

ACM India Summer School on Symmetric Key Cryptography

Nitin Kumar Sharma

Centre for Cryptography and Cybersecurity
IIT Hyderabad

June 13, 2026

Introduction to Stream Ciphers

Introduction- Stream Ciphers

A stream cipher encrypts a message by combining the plaintext with a pseudorandom keystream, usually using the XOR operation:

$$y_i = x_i \oplus k_i$$

where x_i is a plaintext bit, k_i is a keystream bit, and y_i is the resulting ciphertext bit. Decryption is identical since XOR is its own inverse.

Introduction- Stream Ciphers

A stream cipher encrypts a message by combining the plaintext with a pseudorandom keystream, usually using the XOR operation:

$$y_i = x_i \oplus k_i$$

where x_i is a plaintext bit, k_i is a keystream bit, and y_i is the resulting ciphertext bit. Decryption is identical since XOR is its own inverse.

It uses a secret key and usually an initialization vector (IV) or nonce to generate the keystream, ensuring secure and efficient encryption.

Introduction- Stream Ciphers

A stream cipher encrypts a message by combining the plaintext with a pseudorandom keystream, usually using the XOR operation:

$$y_i = x_i \oplus k_i$$

where x_i is a plaintext bit, k_i is a keystream bit, and y_i is the resulting ciphertext bit. Decryption is identical since XOR is its own inverse.

It uses a secret key and usually an initialization vector (IV) or nonce to generate the keystream, ensuring secure and efficient encryption.

Stream ciphers are widely used in real-time communication systems because of their high speed, low hardware complexity, and suitability for continuous data streams.

Advantages

- Simple and efficient (hardware and software).
- Suitable for real-time and constrained environments.

ARX

ARX

Addition $\boxplus$,

ARX

Addition $\boxplus$, Rotation $\lll / \ggg$,

ARX

Addition $\boxplus$, Rotation $\lll / \ggg$, and XOR $\oplus$.

- Uses basic CPU-friendly operations (addition, rotation, XOR).

ARX

Addition $\boxplus$, Rotation $\lll / \ggg$, and XOR $\oplus$.

- Uses basic CPU-friendly operations (addition, rotation, XOR).
- No Lookup tables.

ARX

Addition $\boxplus$, Rotation $\lll / \ggg$, and XOR $\oplus$.

- Uses basic CPU-friendly operations (addition, rotation, XOR).
- No Lookup tables.
- Simple & less operations $\rightarrow$ Fast, less secure.

ARX

Addition $\boxplus$, Rotation $\lll / \ggg$, and XOR $\oplus$.

- Uses basic CPU-friendly operations (addition, rotation, XOR).
- No Lookup tables.
- Simple & less operations $\rightarrow$ Fast, less secure.
- Complex & more operations $\rightarrow$ Secure, but slow.

ARX Ciphers

ARX ciphers are cryptographic algorithms that rely solely on three simple operations on machine words:

- a) Addition modulo 2^{32} (or another word size).
- b) Bitwise rotation.
- c) Bitwise XOR.

The ARX design paradigm offers several benefits:

- **Efficiency:** Extremely fast on modern CPUs since addition, rotation, and XOR are directly supported by most processors.

ARX Ciphers

ARX ciphers are cryptographic algorithms that rely solely on three simple operations on machine words:

- a) Addition modulo 2^{32} (or another word size).
- b) Bitwise rotation.
- c) Bitwise XOR.

The ARX design paradigm offers several benefits:

- **Efficiency:** Extremely fast on modern CPUs since addition, rotation, and XOR are directly supported by most processors.
- **Software-friendliness:** No need for S-boxes or lookup tables, making implementations compact and easy to port.

ARX Ciphers

ARX ciphers are cryptographic algorithms that rely solely on three simple operations on machine words:

- a) Addition modulo 2^{32} (or another word size).
- b) Bitwise rotation.
- c) Bitwise XOR.

The ARX design paradigm offers several benefits:

- **Efficiency:** Extremely fast on modern CPUs since addition, rotation, and XOR are directly supported by most processors.
- **Software-friendliness:** No need for S-boxes or lookup tables, making implementations compact and easy to port.
- **Side-channel resistance:** Avoids cache-timing and table-based side-channel attacks due to the absence of memory lookups.

ARX Ciphers

ARX ciphers are cryptographic algorithms that rely solely on three simple operations on machine words:

- a) Addition modulo 2^{32} (or another word size).
- b) Bitwise rotation.
- c) Bitwise XOR.

The ARX design paradigm offers several benefits:

- **Efficiency:** Extremely fast on modern CPUs since addition, rotation, and XOR are directly supported by most processors.
- **Software-friendliness:** No need for S-boxes or lookup tables, making implementations compact and easy to port.
- **Side-channel resistance:** Avoids cache-timing and table-based side-channel attacks due to the absence of memory lookups.
- **Cryptanalytic resistance:** The interaction of modular addition and XOR complicates classical linear and differential attacks.

ARX Ciphers-Block Ciphers

- **FEAL (1987):** Fast Data Encipherment Algorithm. Proposed as a fast software alternative to DES.

ARX Ciphers-Block Ciphers

- **FEAL (1987):** Fast Data Encipherment Algorithm. Proposed as a fast software alternative to DES.
- **IDEA (1991):** International Data Encryption Algorithm. Widely deployed in email encryption.

ARX Ciphers-Block Ciphers

- **FEAL (1987):** Fast Data Encipherment Algorithm. Proposed as a fast software alternative to DES.
- **IDEA (1991):** International Data Encryption Algorithm. Widely deployed in email encryption.
- **TEA (1994):** Tiny Encryption Algorithm. Used in software.

ARX Ciphers-Block Ciphers

- **FEAL (1987):** Fast Data Encipherment Algorithm. Proposed as a fast software alternative to DES.
- **IDEA (1991):** International Data Encryption Algorithm. Widely deployed in email encryption.
- **TEA (1994):** Tiny Encryption Algorithm. Used in software.
- **XTEA (1997):** Extended Tiny Encryption Algorithm. Improved key schedule structure over TEA.

ARX Ciphers-Block Ciphers

- **FEAL (1987):** Fast Data Encipherment Algorithm. Proposed as a fast software alternative to DES.
- **IDEA (1991):** International Data Encryption Algorithm. Widely deployed in email encryption.
- **TEA (1994):** Tiny Encryption Algorithm. Used in software.
- **XTEA (1997):** Extended Tiny Encryption Algorithm. Improved key schedule structure over TEA.
- **HIGHT (2006):** High performance encryption algorithm for Telematics. Lightweight block cipher.

ARX Ciphers-Block Ciphers

- **FEAL (1987):** Fast Data Encipherment Algorithm. Proposed as a fast software alternative to DES.
- **IDEA (1991):** International Data Encryption Algorithm. Widely deployed in email encryption.
- **TEA (1994):** Tiny Encryption Algorithm. Used in software.
- **XTEA (1997):** Extended Tiny Encryption Algorithm. Improved key schedule structure over TEA.
- **HIGHT (2006):** High performance encryption algorithm for Telematics. Lightweight block cipher.
- **SPECK (2013):** Proposed by the NSA; extensively analyzed in the open literature.

- **Salsa20 (2005):** Selected as a finalist in the eSTREAM project.

ARX Ciphers-Stream Ciphers

- **Salsa20 (2005):** Selected as a finalist in the eSTREAM project.
- **ChaCha (2008):** Used extensively commercially. (Google, Mozilla, etc.)

- **Salsa20 (2005):** Selected as a finalist in the eSTREAM project.
- **ChaCha (2008):** Used extensively commercially. (Google, Mozilla, etc.)
- **BLAKE (2012):** SHA-3 finalist; widely used as a hash function.

ARX Ciphers-Stream Ciphers

- **Salsa20 (2005):** Selected as a finalist in the eSTREAM project.
- **ChaCha (2008):** Used extensively commercially. (Google, Mozilla, etc.)
- **BLAKE (2012):** SHA-3 finalist; widely used as a hash function.
- **Chaskey (2014):** ARX-based message authentication code standardized in ISO/IEC 29192-6 for lightweight cryptography.

ARX Ciphers-Stream Ciphers

- **Salsa20 (2005)**: Selected as a finalist in the eSTREAM project.
- **ChaCha (2008)**: Used extensively commercially. (Google, Mozilla, etc.)
- **BLAKE (2012)**: SHA-3 finalist; widely used as a hash function.
- **Chaskey (2014)**: ARX-based message authentication code standardized in ISO/IEC 29192-6 for lightweight cryptography.
- **Forró (2023)**: Similar to **Salsa20** and **ChaCha**, but with faster diffusion.

- 1 Salsa20
 - Salsa20-Structure
 - Salsa-Round Functions
 - Salsa-Design

- 2 ChaCha
 - ChaCha-Structure
 - ChaCha-Round Functions
 - ChaCha-Design

- 3 Toy-ChaCha

- **Salsa20** is an ARX operation-based cipher introduced by Daniel J. Bernstein [Ber08b] in 2005.

- **Salsa20** is an ARX operation-based cipher introduced by Daniel J. Bernstein [Ber08b] in 2005.
- **Salsa20** works on **sixteen 32-bit words** represented in the form of a 4×4 matrix.

- **Salsa20** is an ARX operation-based cipher introduced by Daniel J. Bernstein [Ber08b] in 2005.
- **Salsa20** works on **sixteen 32-bit words** represented in the form of a 4×4 matrix.
- The cipher has two versions each, having keys of sizes 128 and 256 bits.

- **Salsa20** is an ARX operation-based cipher introduced by Daniel J. Bernstein [Ber08b] in 2005.
- **Salsa20** works on **sixteen 32-bit words** represented in the form of a 4×4 matrix.
- The cipher has two versions each, having keys of sizes 128 and 256 bits.
- The **256-bit** keyversion of the cipher takes 8 key words ($\mathbf{k}_0, \mathbf{k}_1, \dots, \mathbf{k}_7$), 4 constants words ($\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3$), 2 words of nonce ($\mathbf{v}_0, \mathbf{v}_1$) and 2 counter words ($\mathbf{t}_0, \mathbf{t}_1$) as input and generates a **512-bit** output.

- **Salsa20** is an ARX operation-based cipher introduced by Daniel J. Bernstein [Ber08b] in 2005.
- **Salsa20** works on **sixteen 32-bit words** represented in the form of a 4×4 matrix.
- The cipher has two versions each, having keys of sizes 128 and 256 bits.
- The **256-bit** keyversion of the cipher takes 8 key words ($\mathbf{k}_0, \mathbf{k}_1, \dots, \mathbf{k}_7$), 4 constants words ($\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3$), 2 words of nonce ($\mathbf{v}_0, \mathbf{v}_1$) and 2 counter words ($\mathbf{t}_0, \mathbf{t}_1$) as input and generates a **512-bit** output.
- For simplicity, the nonces and counter words are considered as $\mathcal{IV}$'s. The constants words ($\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3$) are slightly different for 128-bit and 256-bit key versions of cipher.

- The four constants for the 256-bit key structure are

$c_0 = \text{0x61707865}$, $c_1 = \text{0x3320646e}$, $c_2 = \text{0x79622d32}$, $c_3 = \text{0x6b206574}$.

- The four constants for the 256-bit key structure are

$$c_0 = \mathbf{0x61707865}, c_1 = \mathbf{0x3320646e}, c_2 = \mathbf{0x79622d32}, c_3 = \mathbf{0x6b206574}.$$

- For the 128-bit key structure, the key words $(\mathbf{k}_0, \mathbf{k}_1, \dots, \mathbf{k}_7)$ are defined as $\mathbf{k}_{i+4} = \mathbf{k}_i, \forall 0 \leq i \leq 3$. There is a slight change in the constants for the 128-bit key structure.

- The four constants for the 256-bit key structure are

$$c_0 = \text{0x61707865}, c_1 = \text{0x3320646e}, c_2 = \text{0x79622d32}, c_3 = \text{0x6b206574}.$$

- For the 128-bit key structure, the key words $(k_0, k_1, \dots, k_7)$ are defined as $k_{i+4} = k_i, \forall 0 \leq i \leq 3$. There is a slight change in the constants for the 128-bit key structure.

- The four constants for the 128-bit key structure are

$$c_0 = \text{0x61707865}, c_1 = \text{0x3120646e}, c_2 = \text{0x79622d36}, c_3 = \text{0x6b206574}$$

- The four constants for the 256-bit key structure are

$$c_0 = \mathbf{0x61707865}, c_1 = \mathbf{0x3320646e}, c_2 = \mathbf{0x79622d32}, c_3 = \mathbf{0x6b206574}.$$

- For the 128-bit key structure, the key words $(\mathbf{k}_0, \mathbf{k}_1, \dots, \mathbf{k}_7)$ are defined as $\mathbf{k}_{i+4} = \mathbf{k}_i, \forall 0 \leq i \leq 3$. There is a slight change in the constants for the 128-bit key structure.

- The four constants for the 128-bit key structure are

$$c_0 = \mathbf{0x61707865}, c_1 = \mathbf{0x3120646e}, c_2 = \mathbf{0x79622d36}, c_3 = \mathbf{0x6b206574}$$

- The keywords $(\mathbf{k}_0, \mathbf{k}_1, \dots, \mathbf{k}_7)$ are arranged in all the rows of the matrix. Nonces $(\mathbf{v}_0, \mathbf{v}_1)$ and counter words $(\mathbf{t}_0, \mathbf{t}_1)$ are positioned in the second and third rows, respectively. The constants words $(\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3)$ are placed along the diagonals.

Based on the discussion above, the initial state matrix looks as follows:

State Matrix

$$X = \begin{pmatrix} X_0 & X_1 & X_2 & X_3 \\ X_4 & X_5 & X_6 & X_7 \\ X_8 & X_9 & X_{10} & X_{11} \\ X_{12} & X_{13} & X_{14} & X_{15} \end{pmatrix} = \begin{pmatrix} c_0 & k_0 & k_1 & k_2 \\ k_3 & c_1 & v_0 & v_1 \\ t_0 & t_1 & c_2 & k_4 \\ k_5 & k_6 & k_7 & c_3 \end{pmatrix}.$$

Based on the discussion above, the initial state matrix looks as follows:

State Matrix

$$X = \begin{pmatrix} X_0 & X_1 & X_2 & X_3 \\ X_4 & X_5 & X_6 & X_7 \\ X_8 & X_9 & X_{10} & X_{11} \\ X_{12} & X_{13} & X_{14} & X_{15} \end{pmatrix} = \begin{pmatrix} c_0 & k_0 & k_1 & k_2 \\ k_3 & c_1 & v_0 & v_1 \\ t_0 & t_1 & c_2 & k_4 \\ k_5 & k_6 & k_7 & c_3 \end{pmatrix}.$$

Round Function QR_s

$$\begin{aligned} b' &= b \oplus ((a \boxplus d) \lll 7); & c' &= c \oplus ((b' \boxplus a) \lll 9); \\ d' &= d \oplus ((c' \boxplus b') \lll 13); & a' &= a \oplus ((d' \boxplus c') \lll 18). \end{aligned} \tag{1}$$

Keystream block Z as $Z = X + X^{(n)}$.

Odd round-Column round

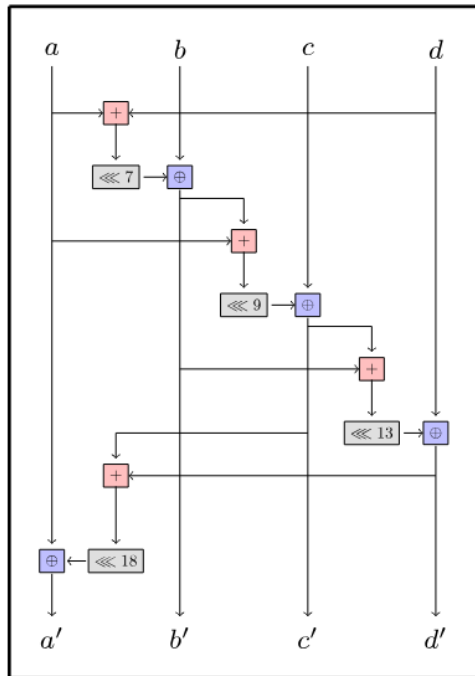
$$QR_s(x[0], x[4], x[8], x[12]);$$
$$QR_s(x[5], x[9], x[13], x[1]);$$
$$QR_s(x[10], x[14], x[2], x[6]);$$
$$QR_s(x[15], x[3], x[7], x[11]);$$

Odd round-Column round

$$QR_s(x[0], x[4], x[8], x[12]);$$
$$QR_s(x[5], x[9], x[13], x[1]);$$
$$QR_s(x[10], x[14], x[2], x[6]);$$
$$QR_s(x[15], x[3], x[7], x[11]);$$

Even round- Row round

$$QR_s(x[0], x[1], x[2], x[3]);$$
$$QR_s(x[5], x[6], x[7], x[4]);$$
$$QR_s(x[10], x[11], x[8], x[9]);$$
$$QR_s(x[15], x[12], x[13], x[14]);$$



- 1 Salsa20
 - Salsa20-Structure
 - Salsa-Round Functions
 - Salsa-Design

- 2 ChaCha
 - ChaCha-Structure
 - ChaCha-Round Functions
 - ChaCha-Design

- 3 Toy-ChaCha

- **ChaCha** was introduced by Daniel J. Bernstein [Ber08a] in 2008.

- **ChaCha** was introduced by Daniel J. Bernstein [Ber08a] in 2008.
- **ChaCha** works on **sixteen 32-bit words** represented in the form of a 4×4 matrix.

- **ChaCha** was introduced by Daniel J. Bernstein [Ber08a] in 2008.
- **ChaCha** works on **sixteen 32-bit words** represented in the form of a 4×4 matrix.
- The cipher has two versions each, having keys of sizes 128 and 256 bits.

- **ChaCha** was introduced by Daniel J. Bernstein [Ber08a] in 2008.
- **ChaCha** works on **sixteen 32-bit words** represented in the form of a 4×4 matrix.
- The cipher has two versions each, having keys of sizes 128 and 256 bits.
- The **256-bit** keyversion of the cipher takes 8 key words ($\mathbf{k}_0, \mathbf{k}_1, \dots, \mathbf{k}_7$), 4 constants words ($\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3$), 2 words of nonce ($\mathbf{v}_0, \mathbf{v}_1$) and 2 counter words ($\mathbf{t}_0, \mathbf{t}_1$) as input and generates a **512-bit** output.

- **ChaCha** was introduced by Daniel J. Bernstein [Ber08a] in 2008.
- **ChaCha** works on **sixteen 32-bit words** represented in the form of a 4×4 matrix.
- The cipher has two versions each, having keys of sizes 128 and 256 bits.
- The **256-bit** keyversion of the cipher takes 8 key words ($\mathbf{k}_0, \mathbf{k}_1, \dots, \mathbf{k}_7$), 4 constants words ($\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3$), 2 words of nonce ($\mathbf{v}_0, \mathbf{v}_1$) and 2 counter words ($\mathbf{t}_0, \mathbf{t}_1$) as input and generates a **512-bit** output.
- For simplicity, the nonces and counter words are considered as $\mathcal{IV}$ s. The constants words ($\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3$) are slightly different for 128-bit and 256-bit key versions of cipher.

- The four constants for the 256-bit key structure are

$c_0 = 0x61707865, c_1 = 0x3320646e, c_2 = 0x79622d32, c_3 = 0x6b206574.$

- The four constants for the 256-bit key structure are

$$c_0 = \text{0x61707865}, c_1 = \text{0x3320646e}, c_2 = \text{0x79622d32}, c_3 = \text{0x6b206574}.$$

- For the 128-bit key structure, the key words $(k_0, k_1, \dots, k_7)$ are defined as $k_{i+4} = k_i, \forall 0 \leq i \leq 3$. There is a slight change in the constants for the 128-bit key structure.

- The four constants for the 256-bit key structure are

$$c_0 = \text{0x61707865}, c_1 = \text{0x3320646e}, c_2 = \text{0x79622d32}, c_3 = \text{0x6b206574}.$$

- For the 128-bit key structure, the key words ($k_0, k_1, \dots, k_7$) are defined as $k_{i+4} = k_i, \forall 0 \leq i \leq 3$. There is a slight change in the constants for the 128-bit key structure.

- The four constants for the 128-bit key structure are

$$c_0 = \text{0x61707865}, c_1 = \text{0x3120646e}, c_2 = \text{0x79622d36}, c_3 = \text{0x6b206574}$$

- The four constants for the 256-bit key structure are

$$c_0 = \text{0x61707865}, c_1 = \text{0x3320646e}, c_2 = \text{0x79622d32}, c_3 = \text{0x6b206574}.$$

- For the 128-bit key structure, the key words ($k_0, k_1, \dots, k_7$) are defined as $k_{i+4} = k_i, \forall 0 \leq i \leq 3$. There is a slight change in the constants for the 128-bit key structure.

- The four constants for the 128-bit key structure are

$$c_0 = \text{0x61707865}, c_1 = \text{0x3120646e}, c_2 = \text{0x79622d36}, c_3 = \text{0x6b206574}$$

- The keywords ($k_0, k_1, \dots, k_7$) are arranged in the second and third rows of the matrix. Nonces (v_0, v_1, v_2) and counter words (t_0) are positioned in the last rows. The constants words (c_0, c_1, c_2, c_3) are placed in the first row.

ChaCha-Structure

Based on the discussion above, the initial state matrix looks as follows:

State Matrix

$$X = \begin{pmatrix} X_0 & X_1 & X_2 & X_3 \\ X_4 & X_5 & X_6 & X_7 \\ X_8 & X_9 & X_{10} & X_{11} \\ X_{12} & X_{13} & X_{14} & X_{15} \end{pmatrix} = \begin{pmatrix} c_0 & c_1 & c_2 & c_3 \\ k_0 & k_1 & k_2 & k_3 \\ k_4 & k_5 & k_6 & k_7 \\ t_0 & v_0 & v_1 & v_2 \end{pmatrix}.$$

ChaCha-Structure

Based on the discussion above, the initial state matrix looks as follows:

State Matrix

$$X = \begin{pmatrix} X_0 & X_1 & X_2 & X_3 \\ X_4 & X_5 & X_6 & X_7 \\ X_8 & X_9 & X_{10} & X_{11} \\ X_{12} & X_{13} & X_{14} & X_{15} \end{pmatrix} = \begin{pmatrix} c_0 & c_1 & c_2 & c_3 \\ k_0 & k_1 & k_2 & k_3 \\ k_4 & k_5 & k_6 & k_7 \\ t_0 & v_0 & v_1 & v_2 \end{pmatrix}.$$

Round Function QR_c

$$\begin{aligned} a' &= a \boxplus b; & d' &= ((d \oplus a') \lll 16); \\ c' &= c \boxplus d'; & b' &= ((b \oplus c') \lll 12); \\ a'' &= a' \boxplus b'; & d'' &= ((d' \oplus a'') \lll 8); \\ c'' &= c' \boxplus d''; & b'' &= ((b' \oplus c'') \lll 7). \end{aligned} \tag{2}$$

Keystream block Z as $Z = X + X^{(n)}.$

Odd round-Column round

$$QR_c(x[0], x[4], x[8], x[12]);$$
$$QR_c(x[1], x[5], x[9], x[13]);$$
$$QR_c(x[2], x[6], x[10], x[14]);$$
$$QR_c(x[3], x[7], x[11], x[15]);$$

Odd round-Column round

$QR_c(x[0], x[4], x[8], x[12]);$

$QR_c(x[1], x[5], x[9], x[13]);$

$QR_c(x[2], x[6], x[10], x[14]);$

$QR_c(x[3], x[7], x[11], x[15]);$

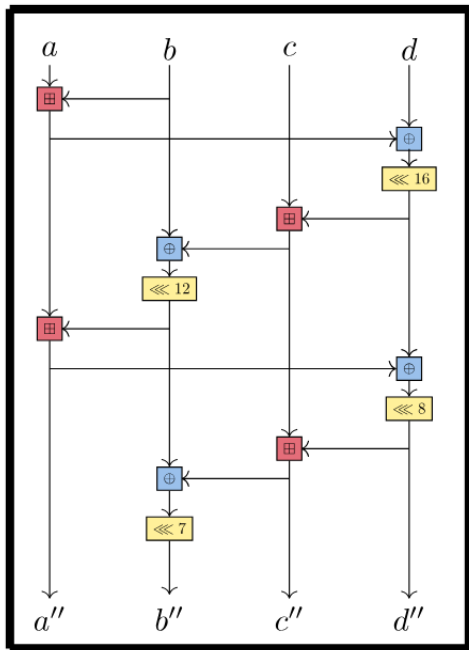
Even round-Diagonal round

$QR_c(x[0], x[5], x[10], x[15]);$

$QR_c(x[1], x[6], x[11], x[12]);$

$QR_c(x[2], x[7], x[8], x[13]);$

$QR_c(x[3], x[4], x[9], x[14]);$



Each entry (word) of the matrix is of 8 bits.

Round Function QR_{tc}

$$\begin{aligned}a' &= a \boxplus b; & d' &= ((d \oplus a') \lll 4); \\c' &= c \boxplus d'; & b' &= ((b \oplus c') \lll 3); \\a'' &= a' \boxplus b'; & d'' &= ((d' \oplus a'') \lll 2); \\c'' &= c' \boxplus d''; & b'' &= ((b' \oplus c'') \lll 1).\end{aligned}\tag{3}$$

Test Vectors

$$X = \begin{pmatrix} 00 & 00 & 00 & 00 \\ 65 & 00 & 00 & 00 \\ 01 & 00 & 00 & 00 \\ ff & ff & ff & ff \end{pmatrix} = \begin{pmatrix} 00 & 00 & 00 & 00 \\ 17 & d3 & 5b & 05 \\ 19 & b5 & 5c & 0c \\ c6 & b1 & 68 & 59 \end{pmatrix}.$$

Constants

$c0 = 0x65, c1 = 0x6e, c2 = 0x32, c3 = 0x74.$

Test Vectors

$$X = \begin{pmatrix} 00 & 00 & 00 & 00 \\ 65 & 00 & 00 & 00 \\ 01 & 00 & 00 & 00 \\ ff & ff & ff & ff \end{pmatrix} = \begin{pmatrix} 00 & 00 & 00 & 00 \\ 17 & d3 & 5b & 05 \\ 19 & b5 & 5c & 0c \\ c6 & b1 & 68 & 59 \end{pmatrix}.$$

Constants

$c0 = 0x65, c1 = 0x6e, c2 = 0x32, c3 = 0x74.$

(ID,OD) as $\Delta_{13}^{(0)}[0], \Delta_1^{(2)}[6]$, with prob $p = 0.7$, bias $c_d = 0.971$.



Daniel J. Bernstein.

ChaCha, a variant of Salsa20, 2008.

Document ID: 4027b5256e17b9796842e6d0f68b0b5e.



Daniel J. Bernstein.

The salsa20 family of stream ciphers.

In *New Stream Cipher Designs - The eSTREAM Finalists*, volume 4986 of *Lecture Notes in Computer Science*, pages 84–97. Springer, 2008.

Questions?